



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Ispravljanje grešaka u tekstu na srpskom jeziku: poređenje rečničkog pristupa, n-gram jezičkog modela i velikog jezičkog modela.

Student:

Natalija Pavlović 2125

Mentor:

dr. Suzana Stojković

Niš, 2026. godina

Sadržaj

1.	Uvod	2
2.	Metode za ispravljanje grešaka u tekstu	4
2.1.	Morfološka obrada teksta i problem ispravljanja grešaka	5
2.2.	Edit rastojanje	5
2.3.	Norvigov algoritam i model kanala sa šumom	7
2.4.	Statistički jezički modeli: n-grami	8
2.5.	Veliki jezički modeli	9
3.	Specifičnosti srpskog jezika	9
3.1.	Dva pisma i transliteracija	10
3.2.	Dijakritici i „ošišana latinica"	10
3.3.	Bogata morfologija	11
3.4.	Resursna ograničenost	11
3.5.	Ekavski i ijekavski izgovor	12
3.6.	Transkripcija stranih imena	12
4.	Metodologija	13
4.1.	Korpus i priprema podataka	14
4.2.	Generisanje sintetičkog test skupa	15
4.3.	Rečnik i edit rastojanje (Norvigov algoritam).....	16
4.4.	N-gram rerangiranje kandidata	17
4.5.	Veliki jezički model	17
4.6.	Metrike evaluacije.....	18
5.	Rezultati.....	19
5.1.	Zbirni rezultati	20
5.2.	Rezultati po tipovima grešaka	22
5.3.	Statistička značajnost razlika	23
6.	Diskusija i analiza grešaka	24
6.1.	Šta ume samo veliki jezički model.....	25
6.2.	Vrednost konteksta: n-gram protiv unigrama	25
6.3.	Gde ne uspeva niko	26

6.4. Prekomerno ispravljanje LLM-a.....	26
6.5. Praktične implikacije.....	26
7. Zaključak.....	27
8. Literatura	28

1. Uvod

Greške u kucanju su sveprisutna pojava u tekstovima koje ljudi proizvode: nastaju usled brzine kucanja, nepoznavanja pravopisa, ograničenja ulaznih uređaja ili jednostavne nepažnje.

Automatsko otkrivanje i ispravljanje ovakvih grešaka jedan je od najstarijih zadataka obrade prirodnih jezika, sa prvim sistemima koji datiraju još iz šezdesetih godina prošlog veka. Uprkos svojoj dugoj istoriji, ovaj zadatak je i danas aktuelan, kako u praktičnim primenama (uređivači teksta, pretraživači, tastature na mobilnim telefonima), tako i kao ogledni primer na kome se jasno vidi evolucija metoda obrade prirodnih jezika: od pravila i rečnika, preko statističkih modela, do savremenih velikih jezičkih modela.

Za srpski jezik ovaj problem ima i dodatnu specifičnu dimenziju. Srpska latinica sadrži pet slova sa dijakritičkim znacima (č, ć, š, ž, đ) koja se u neformalnoj digitalnoj komunikaciji vrlo često zamenjuju svojim osnovnim oblicima bez dijakritika (c, s, z, dj). To je pojava kolokvijalno poznata kao „ošišana latinica". Tekst pisan na taj način je za čitaoca uglavnom razumljiv, ali je formalno pun pravopisnih grešaka. U takvom tekstu neke reči postaju višesmislene (tekst „sef" može odgovarati rečima „sef" i „šef", a „cela" rečima „cela" i „ćela"). Vraćanje dijakritika je stoga poseban i praktično veoma relevantan podzadatak ispravljanja grešaka u srpskom tekstu i on zahteva razmatranje konteksta u kome se reč nalazi.

Predmet ovog rada je izgradnja i poređenje tri pristupa automatskog ispravljanja grešaka u tekstovima na srpskom jeziku:

- rečnički pristup sa edit rastojanjem (Norvigov algoritam): greška se otkriva proverom reči u frekvencijskom rečniku, a ispravka bira među rečima na malom rastojanju izmene, po kriterijumu učestalosti u korpusu
- n-gram jezički model: isti mehanizam detekcije i generisanja kandidata, ali se najbolja ispravka bira na osnovu konteksta, tj. verovatnoće da se kandidat pojavi između susednih reči u rečenici procenjene bigram modelom
- veliki jezički model (LLM): savremeni pristup u kome se rečenica sa greškama prosleđuje unapred istreniranom velikom jezičkom modelu sa uputstvom (promptom) da vrati ispravljenu rečenicu, bez ikakvog dodatnog treniranja (zero-shot režim)

Sva tri pristupa evaluirana su na istom, sintetički generisanom test skupu od 2.000 rečenica sa srpske Vikipedije. U taj skup su programski ubačene pet vrste grešaka:

- uklanjanje dijakritika
- zamena susednim tasterom
- brisanje
- umetanje
- transpozicija znakova.

Pored ukupne tačnosti ispravljanja analizirane su i tačnosti po tipu greške, sposobnost detekcije, sklonost lažnim ispravkama i brzina rada, a statistička značajnost razlika proverena je Meknemarovim testom.

Ovakvo poređenje kvantifikuje koliko svaki nivo dodatne složenosti zaista doprinosi kvalitetu ispravljanja, a ujedno otkriva i klase grešaka na kojima pojedini pristupi sistematski greše, sa posebnim osvrtom na specifičnosti srpskog jezika: bogatu morfologiju, dva pisma i problem dijakritika.

U ovom radu izložene su sledeće teme:

- Teorijske osnove: morfološka obrada teksta, edit rastojanje, statistički jezički modeli i veliki jezički modeli.
- Razmatraju se specifičnosti srpskog jezika od značaja za ovaj zadatak.
- Detaljno se opisuje metodologija: korpus, postupak generisanja test skupa, implementaciju sva tri pristupa i metrike evaluacije.
- Prikazuju se rezultati i njihova diskusija i kvalitativna analiza grešaka.

2. Metode za ispravljanje grešaka u tekstu

Automatsko ispravljanje grešaka u tekstu obuhvata dva logički odvojena zadatka:

- detekciju - utvrđivanje koje reči u tekstu su pogrešne
- korekciju - izbor ispravne reči kojom pogrešnu treba zameniti

Različiti pristupi ovom problemu razlikuju se upravo po tome kojim znanjem se raspolaže pri rešavanju ta dva podzadatka. Najjednostavniji pristupi koriste samo spisak poznatih reči i meru sličnosti, statistički pristupi dodaju znanje o tome koje se reči obično javljaju zajedno, dok savremeni neuronski modeli raspolažu širokim znanjem o jeziku stečenim treniranjem na ogromnim količinama teksta.

2.1. Morfološka obrada teksta i problem ispravljanja grešaka

Pre bilo kakve analize, tekst je potrebno rastaviti na osnovne jedinice - proces poznat kao tokenizacija. Token je najčešće reč, broj ili znak interpunkcije. Tokenizacija se u praksi najčešće realizuje regularnim izrazima. To su obrasci koji opisuju šta se smatra rečju (neprekinuti niz slova, eventualno sa unutrašnjom crticom ili apostrofom), a šta zasebnim simbolom. Tokenizacija za jezike sa dijakritičkim znacima zahteva posebnu pažnju. Kod njih regularni izraz mora eksplicitno obuhvatiti sva slova azbuke, uključujući č, ć, š, ž i đ, jer se ona u podrazumevanim klasama znakova ASCII opsega ne nalaze.

Nakon tokenizacije osnovna struktura na koju se oslanjaju klasični sistemi za ispravljanje grešaka jeste rečnik. Rečnik je skup reči koje se smatraju ispravnim. On se može preuzeti iz leksikografskih izvora ili izgraditi automatski iz velikog korpusa teksta, brojanjem pojavljivanja svake reči. Ovako izgrađen frekvencijski rečnik nosi dve vrste informacija: koje reči postoje i koliko su česte. Učestalost je dragocena jer omogućava da se, među više mogućih ispravki, prednost da verovatnijoj reči. Cena automatske izgradnje je što rečnik sadrži sve oblike reči iz korpusa, a to uključuje i greške koje su se u samom korpusu zatekle. To za rezultat može da ima merljive posledice po kvalitet detekcije.

Detekcija greške u najjednostavnijem obliku svodi se na proveru pripadnosti rečniku: reč koja se ne nalazi u rečniku proglašava se greškom (non-word error). Ovakva detekcija po definiciji ne može otkriti grešku koja slučajno proizvodi drugu postojeću reč (real-word error). Ako je umesto tekst „selo“ otkucano „seno“, obe reči su u rečniku i rečnički detektor ne vidi problem. Takve greške mogu se otkriti samo iz konteksta i tek okolne reči i smisao rečenice otkrivaju koja je reč na tom mestu ispravna.

2.2. Edit rastojanje

Kada je greška detektovana postavlja se pitanje kojom je rečju zameniti. Intuitivno ispravka treba da bude reč koja je najbližnja pogrešno napisanoj. Standardna formalizacija te sličnosti je

Levenštajново (edit) rastojanje. Ono predstavlja najmanji broj elementarnih operacija. Elementarne operacije mogu biti:

- umetanje jednog znaka
- brisanje jednog znaka
- zamene jednog znaka drugim
- transpoziciju susednih znakova

Pokazano je da preko 80% tipičnih grešaka u kucanju čini upravo jedna ovakva operacija pa je edit rastojanje moguće rešenje za ovaj zadatak.

Edit rastojanje se efikasno računa dinamičkim programiranjem. Ako su date reči dužina m i n , formira se matrica dimenzija $(m+1) \times (n+1)$ u kojoj ćelija (i, j) sadrži rastojanje između prefiksa dužine i prve reči i prefiksa dužine j druge reči. Prva vrsta i prva kolona inicijalizuju se redom 0, 1, 2, ..., a svaka naredna ćelija računa se kao minimum tri vrednosti: ćelije iznad uvećane za 1 (brisanje), ćelije levo uvećane za 1 (umetanje) i dijagonalne ćelije uvećane za 1 ako se odgovarajući znaci razlikuju, odnosno za 0 ako su jednaki (zamena ili poklapanje). Vrednost u donjem desnom uglu matrice je traženo rastojanje. Tabela 1 prikazuje matricu za reči „voda” i „vrata”, čije je rastojanje 3 (zamena o $\Rightarrow$ r, zamena d $\Rightarrow$ a i umetanje t).

		v	r	a	t	a
	0	1	2	3	4	5
v	1	0	1	2	3	4
o	2	1	1	2	3	4
d	3	2	2	2	3	4
a	4	3	3	2	3	3

Tabela 1. Matrica dinamičkog programiranja za Levenštajново rastojanje između reči „voda” i „vrata”

Osnovna definicija tretira sve operacije jednako ali u praksi sve greške nisu jednako verovatne. Zamena slova susednim tasterom češća je od zamene udaljenim, a izostavljanje dijakritika (č $\Rightarrow$ c) daleko je češće od proizvoljne zamene. Zato postoje uopštenja sa težinama u kojima svaka operacija nosi cenu zavisnu od para znakova. Takva matrica cena u modelu kanala sa šumom igra ulogu modela greške. Damerau-Levenštajново rastojanje dodaje transpoziciju susednih

znakova kao četvrtu elementarnu operaciju upravo zato što je zamena redosleda dva susedna slova jedan od najčešćih oblika greške u kucanju. U ovom radu težine se ne koriste eksplicitno, ali se ista ideja, da su neke transformacije verovatnije od drugih, realizuje kroz prioritete nivoa generisanja kandidata.

Za ispravljanje grešaka najčešće nije potrebno računati rastojanje između pogrešne reči i svake reči rečnika (što bi bilo skupo), već se koristi obrnut postupak: od pogrešne reči se sistematski generišu sve reči na rastojanju 1 (sva brisanja, transpozicije, zamene i umetanja), pa se od njih zadrže samo one koje postoje u rečniku. Za reč dužine k i azbuku od 27 slova skup reči na rastojanju 1 ima približno $55k+27$ elemenata što je sasvim praktično za izračunavanje. Ako na rastojanju 1 nema nijedne poznate reči, postupak se ponavlja još jednom nad svim generisanim rečima čime se dobijaju kandidati na rastojanju 2.

2.3. Norvigov algoritam i model kanala sa šumom

Standardni teorijski okvir za ispravljanje grešaka je model kanala sa šumom (noisy channel model). Pretpostavka je da je pisac nameravao da napiše ispravnu reč c , ali je ona prolaskom kroz „kanal sa šumom” (proces kucanja) iskrivljena u opaženu reč w . Zadatak ispravljača je da pronađe ono c koje maksimizuje uslovnu verovatnoću $P(c|w)$, što se svodi na maksimizaciju proizvoda $P(w|c) \cdot P(c)$. Prvi činilac je model greške (koliko je verovatno da se c iskrivi baš u w), a drugi jezički model (koliko je reč c uopšte verovatna u jeziku).

Norvig je pokazao da se već vrlo gruba aproksimacija ovog modela ponaša dobro. U njegovom algoritmu jezički model $P(c)$ je prosto relativna učestalost reči u korpusu, a model greške se ne procenjuje eksplicitno već se zamenjuje strogim prioritetom po rastojanju. Kandidati na rastojanju 1 uvek imaju prednost nad kandidatima na rastojanju 2, a među kandidatima istog ranga bira se najfrekventniji.

Algoritam se svodi na tri koraka:

1. proveriti da li je reč poznata
2. ako nije, generiši poznate kandidate na rastojanju 1, pa tek ako ih nema, na rastojanju 2
3. vrati kandidata sa najvećom učestalošću u korpusu

Uprkos jednostavnosti (originalna implementacija staje u malo linija koda) ovaj algoritam je decenijama polazna tačka i osnovna referentna vrednost (baseline) za zadatak ispravljanja grešaka.

Ključno ograničenje Norvigovog algoritma je što je jezički model unigramski: kandidat se bira isključivo po sopstvenoj učestalosti, bez ikakvog uvida u ostatak rečenice. Za grešku „radm” kandidati na rastojanju 1 uključuju oblike „rad”, „radi”, „radu” i „radom” unigramski model će

izabrali globalno najčešći od njih iako u rečenici „počela sa radm 2. oktobra” padežni kontekst jasno zahteva instrumental „radom”. Upravo tu prazninu popunjava n-gram jezički model.

2.4. Statistički jezički modeli: n-grami

Jezički model je funkcija koja proizvoljnom nizu reči dodeljuje verovatnoću. Verovatnoća je mera koliko je taj niz prirodan u datom jeziku. Ona rečenice sa lančanim pravilom razlaže na proizvod uslovnih verovatnoća svake reči pri datoj celokupnoj prethodnoj istoriji. Kako je istorija u opštem slučaju predugačka da bi se njene verovatnoće pouzdano procenile uvodi se pretpostavka: verovatnoća reči zavisi samo od prethodnih $n-1$ reči. Tako se dobija n-gram model: za $n=1$ unigramski (reči se posmatraju nezavisno), za $n=2$ bigramski (svaka reč zavisi od prethodne), za $n=3$ trigramski, i tako dalje.

Parametri n-gram modela procenjuju se metodom maksimalne verodostojnosti - prostim brojanjem. Uslovna verovatnoća $P(w_2|w_1)$ procenjuje se kao količnik broja pojavljivanja bigrama (w_1, w_2) i broja pojavljivanja reči w_1 u korpusu. Da bi i prva i poslednja reč rečenice imale kontekst, rečenice se obeležavaju posebnim graničnim simbolima za početak ($\langle s \rangle$) i kraj ($\langle /s \rangle$). Treniranje modela je jedan prolaz kroz korpus uz brojanje bez ikakve optimizacije što n-gram modele čini izuzetno brzim i pogodnim za rad na običnom računaru.

Ako se u korpusu reč „na” javlja 100.000 puta, a bigram „na sat” 200 puta, procena je $P(\text{sat}|\text{na}) = 200/100.000 = 0,002$. Ako se bigram „na sito” javlja svega 2 puta, $P(\text{sito}|\text{na}) = 0,00002$, pa će model pri ispravljanju greške „sato” u kontekstu „pogledao je na sato” stostruko favorizovati kandidata „sat”, iako su oba kandidata na istom edit rastojanju i oba postoje u rečniku. Upravo ova sposobnost razlikovanja kandidata po kontekstu čini n-gram model nadgradnjom unigramskog izbora.

Centralni problem n-gram modela je oskudnost podataka (data sparsity). Ogroman broj sasvim ispravnih kombinacija reči nikada se ne pojavi ni u vrlo velikom korpusu pa bi njihova procenjena verovatnoća bila nula, što obesmišljava poređenje kandidata. Rešenje su tehnike izgladivanja (smoothing), koje deo verovatnosne mase preraspodeljuju na neviđene događaje. Najjednostavnije je aditivno (add-k) izgladivanje koje svakom brojaču dodaje malu konstantu. Postoje i sofisticiranije metode, poput Kacovog ili Knezer-Nej izgladivanja, koje kombinuju modele različitih redova tako što se sa nepouzdanog višeg reda „povlače” (backoff) na niži.

U ovom radu korišćen je stupid backoff. On radi tako što ako je bigram viđen u korpusu koristi se njegova relativna učestalost, ako nije koristi se unigramska verovatnoća pomnožena fiksnim faktorom $\alpha=0,4$ (uz aditivno izgladivanje unigramske osnove). Rezultujuće vrednosti formalno nisu normalizovane verovatnoće, ali za rangiranje kandidata to nije ni potrebno. Metoda ne zahteva podešavanje na izdvojenom skupu podataka, ima konstantno vreme izračunavanja i standardan je izbor za velike n-gram modele. Za primenu na ispravljanje grešaka kandidat se

ocenjuje dvostranim kontekstom: zbirom logaritma verovatnoće kandidata pri datoj prethodnoj reči i logaritma verovatnoće naredne reči pri datom kandidatu, čime se u izbor ispravke uključuju obe susedne reči.

2.5. Veliki jezički modeli

Savremenu generaciju jezičkih modela čine neuronski modeli zasnovani na transformer arhitekturi. Umesto brojanja n-grama, transformer uči guste vektorske reprezentacije reči (word embeddings) i mehanizmom pažnje (attention) za svaku poziciju u nizu izračunava reprezentaciju koja uzima u obzir sve ostale pozicije čime se prevazilazi ograničenje fiksnog, kratkog konteksta n-gram modela. Veliki jezički modeli (Large Language Models, LLM) su transformeri sa milijardama parametara, trenirani na korpusima od više hiljada milijardi tokena na zadatku predviđanja naredne reči, a zatim dodatno prilagođeni da prate uputstva korisnika (instruction tuning).

Jedno od ključnih svojstava LLM-ova je učenjem u kontekstu (in-context learning). Model je u stanju da izvrši zadatak koji mu je opisan običnim jezikom u ulaznom tekstu (promptu) bez ikakvog dodatnog treniranja. Kada se zadatak zadaje samo opisom govori se o zero-shot režimu. Kada se u prompt uključi i nekoliko rešenih primera to je few-shot režimu. Za ispravljanje grešaka to znači da je dovoljno modelu proslediti rečenicu uz uputstvo da vrati njen ispravljen oblik, model se oslanja na znanje o srpskom jeziku stečeno tokom treniranja, koje implicitno objedinjuje i rečnik, i model greške, i jezički model sa praktično neograničenim kontekstom.

Ova snaga dolazi sa specifičnim slabostima. Izlaz LLM-a je slobodan tekst i to znači da model može uprkos uputstvu da prepravi i reči koje nisu bile pogrešne, da promeni interpunkciju ili red reči i da se pojavi prekomerno ispravljanje (overcorrection). Drugo LLM je za nekoliko redova veličine sporiji i računski skuplji od rečničkih i n-gram metoda jer svaki poziv zahteva prolaz kroz neuronsku mrežu sa milijardama parametara. Isto, ponašanje modela na jezicima koji su slabije zastupljeni u podacima za treniranje, kakav je srpski, manje je predvidivo nego na engleskom, pa je empirijska provera na srpskom test skupu utoliko nepredvidivija.

Kvalitet izlaza LLM-a u zero-shot režimu bitno zavisi od formulacije prompta, discipline koja se naziva prompt inženjeringom. Za zadatak ispravljanja grešaka prompt mora precizno odrediti šta model sme da menja (pravopisne i tipografske greške) a šta ne (stil, izbor reči, red reči), kao i format odgovora (samo ispravljena rečenica, bez uvoda i objašnjenja), jer svako odstupanje otežava automatsko poređenje izlaza sa referencom. Few-shot varijanta sa nekoliko rešenih primera u promptu dodatno učvršćuje očekivani format i implicitno pokazuje modelu vrstu grešaka koje treba da traži.

3. Specifičnosti srpskog jezika

Srpski jezik unosi u zadatak ispravljanja grešaka nekoliko osobenosti koje ga čine zahtevnijim od engleskog na kome je većina klasičnih algoritama razvijana i evaluirana.

3.1. Dva pisma i transliteracija

Srpski je redak primer jezika sa punom, živom digrafijom: ravnopravno se piše ćirilicom i latinicom, a prosečan govornik bez napora čita i piše oba pisma. U digitalnom prostoru pisma koegzistiraju bez jasnog pravila. Zvanični i medijski sadržaji često su ćirilični, dok u neformalnoj komunikaciji preovlađuje latinica. Korpusi prikupljeni sa interneta, uključujući i Vikipediju zato po pravilu sadrže mešavinu oba pisma, a ponekad čak i unutar istog dokumenta.

Za izgradnju jedinstvenog rečnika i jezičkog modela neophodno je sav tekst svesti na jedno pismo. U suprotnom bi se statistika svake reči rasparčala na dva nezavisna oblika (npr. „kyħa" i „kuća" brojali bi se odvojeno), čime bi se veštački prepolovila količina podataka po reči. Transliteracija ćirilice u latinicu gotovo je trivijalno preslikavanje znak-na-znak, uz jedan važan izuzetak: slova Љ, Њ i Џ nemaju svoje pojedinačne latinične ekvivalente već se preslikavaju u digrafe lj, nj i dž - parove latiničnih znakova. Digrafi su i razlog zašto je smer transliteracije bitan: preslikavanje ćirilica => latinica je jednoznačno, dok obrnuti smer nije. Latinični niz „nj" može poticati i od slova Њ i od dodira slova n i j na granici morfema (uporediti „njiva" i „injekcija" ili „konjunkcija"). Latinica je zato prirodan izbor ciljnog pisma, a implementacija transliteracije mora digrafe obrađivati pre pojedinačnih slova vodeći računa i o velikim slovima (Јб => Lj).

3.2. Dijakritici i „ošišana latinica"

Pet slova srpske latinice nosi dijakritičke znake: č, ć, š, ž i đ. U neformalnom digitalnom tekstu oni se masovno izostavljaju - č i ć postaju c, š postaje s, ž postaje z, a đ se najčešće piše kao dvoznak dj. Ova praksa, kolokvijalno nazvana „ošišana latinica", istorijski potiče iz vremena SMS poruka i računarskih sistema ograničenih na ASCII skup znakova, ali se zadržala do danas kao navika, i to u tolikoj meri da znatan deo srpskog teksta na internetu uopšte ne sadrži dijakritike.

Sa stanovišta ispravljanja grešaka ošišana latinica je sistematski i predvidiv tip greške, ali sa dve neprijatne osobine. Prva je višesmislenost: pošto se tri različita slova (c, č, ć) stapaju u jedno, ošišani oblik može odgovarati većem broju ispravnih reči, a pravu je moguće izabrati samo iz konteksta - oblik „kuca" može značiti „kuca" (glagol) ili „kuća" (imenica), „sef" može biti „sef" ili „šef", „cela" može biti „cela" ili „ćela".

Druga neprijatnost tiče se edit rastojanja: reč sa više dijakritika u očišanom obliku udaljena je od originala za onoliko zamena koliko je dijakritika izostavljeno (npr. „čišćenje" => „ciscenje" je na rastojanju 4), pa standardno generisanje kandidata na rastojanju 1 i 2 ne dopire do ispravke. Zato je u ovom radu implementiran poseban generator kandidata koji ciljano nabraja sve moguće restauracije dijakritika (svako c širi u {c, č, ć}, svako s u {s, š}, svako z u {z, ž}, a svaki par dj u {dj, đ}) nezavisno od edit rastojanja.

Konačno, raširenost očišanog pisanja ima i težu posledicu: očišani oblici prodiru u same korpus iz kojih se rečnik gradi. Rečnik izgrađen iz stvarnog teksta tako poznaje i pogrešne oblike (u korpusu ovog rada oblik „medju" javlja se dovoljno često da bude uvršten u rečnik), pa ih rečnička detekcija propušta kao ispravne reči.

3.3. Bogata morfologija

Srpski je jezik izrazito bogate morfologije: imenice se menjaju kroz sedam padeža u jednini i množini, pridevi se slažu sa imenicom u rodu, broju i padežu, a glagoli imaju razvijenu konjugaciju po licima, vremenima i vidovima. Jedna imenica tako tipično ima desetak različitih površinskih oblika (npr. rad, rada, radu, radom, radovi, radova, radovima...), a pridevi i po nekoliko desetina.

To onda za ispravljanje grešaka ima dve posledice. Prva je praktična: rečnik izgrađen bez svođenja reči na osnovni oblik (lematizacije) mora sadržati sve oblike svake reči, što ga čini višestruko većim nego za morfološki siromašnije jezike. U ovom radu rečnik broji blizu 589.000 različitih oblika. Lematizacija bi rečnik smanjila, ali bi uništila upravo ono što je ispravljaču potrebno a to je sposobnost da vrati tačan padežni oblik, a ne osnovni. Druga posledica je suštinska: mnogi oblici iste lekseme međusobno su na edit rastojanju 1 pa greška u kucanju često ima više morfoloških kandidata za ispravku koji su svi legitimne reči. Unigramski model među njima bira globalno najčešći oblik što je često pogrešno. U rečenici „počela sa radm 2. oktobra" jedino instrumental „radom" gramatički odgovara, ali je nominativ „rad" frekventniji u korpusu. Bogata morfologija tako čini srpski posebno dobrim testom stvarne vrednosti kontekstualnih modela: razlika između unigramskog i bigramskog izbora kandidata na srpskom je izraženija nego na engleskom gde je imenica najčešće nepromenljiva.

3.4. Resursna ograničenost

Za srpski postoji znatno manje jezičkih resursa nego za velike svetske jezike. Dok za engleski postoje standardni, javno dostupni skupovi parova (pogrešan tekst, ispravan tekst) na kojima se ispravljači decenijama porede, za srpski takav standardni evaluacioni skup ne postoji.

I zastupljenost srpskog u podacima na kojima se treniraju veliki jezički modeli daleko je manja od zastupljenosti engleskog pa se performanse LLM-ova na srpskom ne mogu pretpostaviti na

osnovu rezultata objavljenih za engleski. To evaluaciju na srpskom čini samostalno vrednim empirijskim pitanjem.

U ovom radu test skup je generisan sintetički, kontrolisanim ubacivanjem grešaka u čist tekst. Ovakav pristup ima tri prednosti. Referentne ispravke su potpuno pouzdane jer se za svaku grešku tačno zna koja je reč bila originalna. Broj test primera može se učiniti proizvoljno velikim. Raspodela tipova grešaka je pod punom kontrolom, što omogućava poređenje tačnosti po tipu na uzorcima jednake veličine. Nedostatak sintetičkog pristupa je što veštački generisane greške ne moraju verno odražavati raspodelu stvarnih ljudskih grešaka.

3.5. Ekavski i ijekavski izgovor

Srpski jezik ima dva ravnopravna književna izgovora, ekavski i ijekavski, koji se razlikuju u refleksu praslovenskog glasa jat: reč koja se ekavski piše „delovi” ijekavski glasi „dijelovi”, „lepo” postaje „lijepo”, „reka” postaje „rijeka”. Iako u Srbiji preovlađuje ekavica, tekstovi na internetu, pa i srpska Vikipedija, sadrže i ijekavske oblike, bilo u citatima, bilo u člancima preuzetim ili prilagođenim iz drugih izvora.

Za ispravljanje grešaka ovo znači da rečnik izgrađen iz stvarnog korpusa sadrži oba izgovorna oblika mnogih reči, što dodatno uvećava rečnik i stvara nove parove reči na malom edit rastojanju. Reči „delovi” i „dijelovi” nisu greške jedno drugog već varijante. Sistem za ispravljanje po pravilu ne treba da „ispravlja” izgovornu varijantu, ali frekvencijski kriterijum može da ga navede baš na to. Ako je greška u kucanju bliska i ekavskom i ijekavskom obliku model može izabrati varijantu koja ne odgovara ostatku teksta. U evaluaciji ovog rada oba oblika tretirana su kao ispravna kad god su se poklapala sa referentnom rečju, a prisustvo ijekavskih oblika u test rečenicama (npr. „dijelovi glukoznih lanaca”) ilustruje da je korpus u tom pogledu heterogen.

3.6. Transkripcija stranih imena

Srpski pravopis propisuje fonetsku transkripciju stranih vlastitih imena. Strana imena se ne prenose u izvornom obliku već onako kako se izgovaraju, prilagođeno srpskom glasovnom sistemu. Jeff Bezos postaje „Džef Bezos”, a kompanija Microsof postaje „Majkrosoft”. Uz to tekstovi tehničke i popularne tematike obiluju i netranskribovanim nazivima (Worldcraft) i pozajmljenicama u raznim stepenima prilagođenosti („frajer”, „skiper”, „biper”).

Za ispravljanje grešaka vlastita imena i pozajmljenice su sistematski najteža klasa reči. Retke su pa su njihove frekvencije u korpusu male ili nulte čime frekvencijski kriterijum izbora kandidata gubi pouzdanost. Međutim mnoge su jedinstvene u korpusu pa greška u imenu često nema nijedan validan rečnički kandidat u blizini, a ni kontekst ne pomaže mnogo, jer se ime može

javiti u praktično bilo kom okruženju. Za velike jezičke modele strana imena su problematična iz drugog razloga. Model može prepoznati izvorni oblik imena i vratiti ga u originalnom formatu umesto u srpskoj transkripciji što predstavlja pogrešnu ispravku. Upravo se u ovoj klasi reči koncentrišu slučajevi na kojima ne uspeva nijedan od tri poređena pristupa.

4. Metodologija

Eksperimentalni deo rada realizovan je kao sistem u programskom jeziku Python, organizovan u niz skripti koje redom pripremaju korpus, generišu test skup, primenjuju tri pristupa ispravljanju i evaluiraju rezultate.

Svi lokalni koraci (priprema podataka, rečnički i n-gram pristup) izvršavaju se na običnom računaru, jedino pristup zasnovan na velikom jezičkom modelu koristi spoljašnji API servis. Radi reproduktivnosti svi postupci koji uključuju slučajnost koriste fiksirano seme generatora slučajnih brojeva.

4.1. Korpus i priprema podataka

Kao izvor čistog srpskog teksta korišćen je korpus srpske Vikipedije iz kolekcije Leipzig Corpora Collection izdanje iz 2021. godine sa milion rečenica. Kolekcija isporučuje tekst već segmentiran na rečenice, po jednu u svakom redu, čime je izbegnuta potreba za sopstvenom segmentacijom.

Nad sirovim rečenicama sprovedeno je čišćenje u nekoliko koraka:

- normalizacija razmaka i uklanjanje zaostalih oznaka formatiranja sa početka rečenica
- transliteracija ćiriličnih rečenica u latinicu (uz pravilno rukovanje digrafima lj, nj i dž)
- tokenizacija regularnim izrazom koji obuhvata sva slova srpske latinice uključujući dijakritike, odbacivanje prekratkih (manje od 4 reči) i predugačkih (preko 60 tokena) rečenica, kao i rečenica u kojima reči čine manje od 60% tokena
- uklanjanje duplikata

Ovim postupkom dobijeno je 947.312 čistih rečenica. Osnovne statistike korpusa prikazane su u Tabeli 2.

Veličina	Vrednost
Ukupno očišćenih rečenica	947.312
Rečenice trening skupa	945.312
Izdvojene (held-out) rečenice	2.000
Tokeni trening skupa (sa interpunkcijom)	16.898.421
Reči trening skupa	14.312.654
Veličina rečnika (različitih oblika reči)	588.767

Tabela 2. Statistike korpusa korišćenog za izgradnju rečnika i jezičkog modela.

Iz očišćenog skupa nasumično je izdvojeno 2.000 rečenica koje čine osnovu test skupa i koje ni na koji način nisu korišćene pri izgradnji rečnika i jezičkog modela. Ovo strogo razdvajanje je neophodno da bi evaluacija bila poštena jer bi u suprotnom modelu bili ocenjivani na podacima koje su videli tokom izgradnje. Preostalih 945.312 rečenica čini trening skup sa ukupno oko 16,9 miliona tokena od čega oko 14,3 miliona reči.

4.2. Generisanje sintetičkog test skupa

Test skup generisan je programski ubacivanjem kontrolisanih grešaka u 2.000 izdvojenih rečenica. U svaku rečenicu ubačena je tačno jedna greška, u nasumično izabranu reč dužine najmanje tri znaka (brojevi i interpunkcija se preskaču), pri čemu se za svaku grešku beleži originalna reč, iskvareni oblik, tip greške i pozicija u rečenici. Time se za svaki test primer tačno zna šta je ispravan odgovor pa je evaluacija potpuno automatska.

Implementirano je pet tipova grešaka koji pokrivaju i klasične greške u kucanju i pojam očišćane latinice:

- uklanjanje dijakritika: jedan ili više dijakritičkih znakova u reči zamenjuje se osnovnim oblikom (č, ć => c, š => s, ž => z, đ => dj), čime se simulira očišćana latinica
- zamena susednim tasterom: jedan znak reči zamenjuje se znakom sa susednog tastera na tastaturi
- brisanje: iz reči se uklanja nasumično izabran znak
- umetanje: u reč se uz nasumično izabran znak, umeće znak sa njemu susednog tastera
- transpozicija: dva susedna različita znaka reči zamenjuju mesta

Za zamenu i umetanje potreban je model tastature i to je izgrađena mapa susednosti tastera srpske latinične QWERTZ tastature. Ona je izgrađena iz tri slova reda (qwertyuiopšđ, asdfghjkl čć, yxcvbnm) gde su susedi svakog tastera oni levo i desno u istom redu, kao i najbliži tasteri u redu iznad i ispod. Ovakav model greške verniji je stvarnom kucanju od zamene proizvoljnim slovom azbuke jer osoba koja promaši taster najčešće pogodi neki od susednih.

Da bi tačnost mogla pouzdano da se poredi po tipovima grešaka raspodela tipova mora biti ujednačena. Pošto svi tipovi nisu primenljivi na svaku reč (dijakritička greška zahteva reč koja sadrži bar jedan dijakritik), tip greške za svaku rečenicu biran je pravilom uravnoteženja. Od tipova primenljivih na reči date rečenice bira se onaj koji trenutno najviše zaostaje za ciljnom ravnomernom raspodelom. Rezultat je test skup od 2.000 rečenica sa tačno 400 grešaka svakog od pet tipova (Tabela 3), čime su obezbeđeni jednaki uzorci za poređenje po tipu.

Tip greške	Primer (original => greška)	Broj	Udeo
uklanjanje dijakritika	šetnja => setnja	400	20%
zamena susednim tasterom	selo => seno	400	20%
brisanje	čini => ini	400	20%
umetanje	sastav => sastavb	400	20%
transpozicija	ili => lii	400	20%

Tabela 3. Tipovi sintetičkih grešaka sa primerima iz test skupa i njihova raspodela.

Važno svojstvo ovako generisanog skupa je da pojedine greške slučajno proizvode druge postojeće reči pa tako zamena „selo“ => „seno“ daje sasvim legitimnu reč. Takve greške nisu izbacivane iz skupa jer se javljaju i u stvarnom kucanju. One postavljaju prirodnu gornju granicu odziva detekcije za rečničke pristupe i omogućavaju da se izmeri prednost pristupa koji koriste kontekst.

4.3. Rečnik i edit rastojanje (Norvigov algoritam)

Prvi pristup je implementacija Norvigovog algoritma prilagođena srpskom jeziku. Iz trening skupa izgrađen je frekvencijski rečnik: za svaku reč (svedenu na mala slova) prebrojana su pojavljivanja u korpusu čime je dobijeno 588.767 različitih oblika. Reč se smatra poznatom ako se u rečniku javlja bar jednom. Svaka nepoznata reč u ulaznoj rečenici proglašava se greškom i pokušava se njena ispravka.

Kandidati za ispravku generišu se u pet prioriternih nivoa, a pretraga se zaustavlja na prvom nivou koji sadrži bar jednu poznatu reč:

1. sama reč (ako je poznata, ne dira se)
2. dijakritičke restauracije: sve varijante reči u kojima je svako c zamenjeno nekim od {c, č, ć}, svako s nekim od {s, š}, svako z nekim od {z, ž}, a svaki par dj sa {dj, đ}
3. sve reči na edit rastojanju 1
4. sve reči na edit rastojanju 2
5. kombinacija: dijakritičke restauracije na koje je dodatno primenjeno edit rastojanje 1

Generator na svim nivoima koristi azbuku od 27 slova srpske latinice (bez stranih q, w, x, y) čime se izbegava generisanje kandidata koji nisu srpske reči. Dijakritički nivo je ključna adaptacija za srpski: reč „ciscenje“ je na edit rastojanju 4 od „čišćenje“, nedostižnom za standardne nivoe 3 i 4, ali je dijakritička restauracija nalazi direktnim nabranjem. Redosled nivoa kodira implicitni model greške, pretpostavku da je izostavljanje dijakritika verovatniji uzrok nepoznate reči od tipfelera, a tipfeler od dva tipfelera.

Među kandidatima prvog nepraznog nivoa bira se onaj sa najvećom učestalošću u korpusu, a izabranoj ispravci se vraća kapitalizacija originala (velika početna slova, akronimi). Ceo postupak je čisto rečnički i unigramski: ne koristi nikakvu informaciju o susednim rečima, pa služi kao osnovna referentna vrednost prema kojoj se meri doprinos konteksta u naredna dva pristupa.

4.4. N-gram rerangiranje kandidata

Drugi pristup zadržava detekciju i generisanje kandidata identičnim kao u Pristupu 1, isti frekvencijski rečnik, isti prioritetni nivoi kandidata, isti dijakritički generator, a menja isključivo kriterijum izbora među kandidatima. Ova odluka je namerna: pošto se dva pristupa razlikuju u tačno jednoj komponenti, svaka razlika u rezultatima može se pripisati upravo uvođenju konteksta, a ne nekoj drugoj razlici u implementaciji.

Iz trening skupa izgrađen je bigram jezički model: pored unigramskih brojača (istih kao u rečniku), prebrojani su i svi parovi susednih reči pri čemu je svaka rečenica oivičena graničnim simbolima $\langle s \rangle$ i $\langle /s \rangle$ kako bi i prva i poslednja reč imale definisan kontekst. Model sadrži 588.769 unigramskih i nekoliko miliona bigramskih tipova, a verovatnoće se računaju stupid backoff metodom: relativna učestalost bigrama kad je bigram viđen u korpusu, a u suprotnom unigramska verovatnoća (sa aditivnim izgladivanjem) pomnožena faktorom $\alpha = 0,4$.

Pri ispravljanju za svaku detektovanu grešku najpre se pronalaze najbliža reč levo i najbliža reč desno od nje (preskačući interpunkciju, na granicama rečenice koriste se granični simboli). Svaki kandidat k se zatim ocenjuje dvostranim kontekstnim rezultatom, zbirom $\log P(k \mid \text{prethodna reč})$ i $\log P(\text{naredna reč} \mid k)$, i bira se kandidat sa najvećim rezultatom. Time se u izbor uključuju obe susedne reči: za grešku „radm” u kontekstu „sa radm 2.” bigram „sa radom” nadjačava frekventniji ali kontekstno neuklopljen nominativ „rad”.

Pošto se detekcija i skup kandidata ne menjaju ovaj pristup ne može ispraviti ništa što Pristup 1 ne može ni da detektuje (real-word greške ostaju nevidljive), niti da pronađe kandidata van zajedničkih nivoa generisanja. Njegov potencijalni dobitak je u potpunosti skoncentrisan u kvalitetnijem izboru među istim kandidatima.

4.5. Veliki jezički model

Treći pristup predstavlja ispravljanje sa velikim jezičkim modelom u zero-shot režimu. Svaka rečenica test skupa prosleđuje se modelu kroz API servis uz uputstvo (prompt) na srpskom jeziku koje od modela traži da ispravi isključivo pravopisne i tipografske greške, dijakritike i tipfelere, bez menjanja stila, izbora reči ili reda reči, i da odgovori isključivo ispravljenom rečenicom, bez uvoda, objašnjenja ili navodnika. Ovako restriktivna formulacija prompta je namerna: svaka

sloboda u izlazu (objašnjenja, prepričavanje, dodatne izmene) otežava automatsko poređenje odgovora sa referentnom rečenicom.

Korišćen je sledeći prompt:

Ispravi isključivo pravopisne i tipografske greške u sledećoj rečenici (dijakritike, tipfelere). Ne menjaj stil, reči ni red reči osim da ispraviš greške. Odgovori isključivo ispravljenom rečenicom - bez uvoda, objašnjenja, navodnika ili napomena.

Rečenica: {sentence}

Ispravljena rečenica:

Korišćen je model Llama 3.3 sa 70 milijardi parametara, otvorenih težina, preko Groq API servisa. Implementacija je izvedena kroz uniformni programski interfejs koji omogućava zamenu provajdera (Anthropic, OpenAI, Groq) bez izmene ostatka sistema. API ključevi se čitaju iz promenljivih okruženja (.env) i ne nalaze se u kodu. Pošto svaki API poziv troši vreme i podleže ograničenjima broja zahteva, ugrađena su tri zaštitna mehanizma: automatsko ponavljanje neuspelih poziva sa eksponencijalnim povećanjem pauze, pauza između uzastopnih zahteva i trajno keširanje svih odgovora na disk tako da se prekinuto izvršavanje nastavlja bez ponovnog slanja već obrađenih rečenica.

Za razliku od prva dva pristupa koji vraćaju listu tačno lokalizovanih izmena, LLM vraća slobodan tekst cele rečenice. Da bi se njegov izlaz sveo na isti oblik kao kod ostalih pristupa odgovor se najpre čisti od eventualnih uvodnih fraza i navodnika, a zatim se tokenizuje i poravnava sa ulaznom rečenicom algoritmom za poređenje sekvenci. Blokovi izmena jednake dužine preslikavaju se token-na-token dok se neravni blokovi (kada je model spojio, razdvojio, dodao ili obrisao reči) pripisuju prvoj zahvaćenoj poziciji. Ovim poravnanjem se za svaku ubačenu grešku može utvrditi da li je i kako ispravljena, a svaka izmena reči koja nije bila pogrešna registruje se kao prekomerno ispravljanje.

4.6. Metrike evaluacije

Sva tri pristupa evaluirana su na istom test skupu potpuno automatski. Pošto je za svaku ubačenu grešku poznata originalna reč ispravka se ocenjuje prostim poređenjem (neosetljivim na velika i mala slova). Kvalitet ispravljanja meren je kroz četiri metrike koje razdvajaju dva podzadatka, detekciju i korekciju, i hvataju cenu pogrešnih intervencija:

- odziv detekcije (detection recall): udeo ubačenih grešaka koje je pristup uopšte dirnuo, tj. promenio pogrešnu reč u bilo šta
- tačnost ispravke (correction accuracy): udeo ubačenih grešaka ispravljenih tačno u originalnu reč i ovo je primarna metrika poređenja

- stopa lažnih izmena (false positive rate): udeo originalno ispravnih reči koje je pristup izmenio, kod rečničkih pristupa to su pogrešne ispravke retkih ali ispravnih reči, a kod LLM-a prekomerno ispravljanje
- brzina: broj obrađenih rečenica u sekundi, merena na istom računaru, za LLM pristup brzina odražava latenciju API servisa, a ne lokalno izračunavanje

Tačnost ispravke računata je i zbirno i po tipu greške, što ujednačena raspodela tipova u test skupu (400 grešaka po tipu) čini pouzdanim. Svi tipovi porede se na uzorcima iste veličine.

Za poređenje pristupa nije dovoljno uporediti procenite jer razlika može biti posledica slučajnosti uzorkovanja. Pošto su svi pristupi ocenjivani na istim greškama primenjen je Meknemarov test, upareni test za poređenje dva klasifikatora na zajedničkom uzorku. Test posmatra samo diskordantne parove, greške koje je jedan pristup ispravio a drugi nije i proverava da li je njihova raspodela značajno asimetrična. Korišćena je egzaktna dvostrana varijanta testa sa pragom značajnosti 0,05. Mala p-vrednost znači da uočena razlika u tačnosti nije verovatna pod pretpostavkom da su pristupi jednako dobri.

5. Rezultati

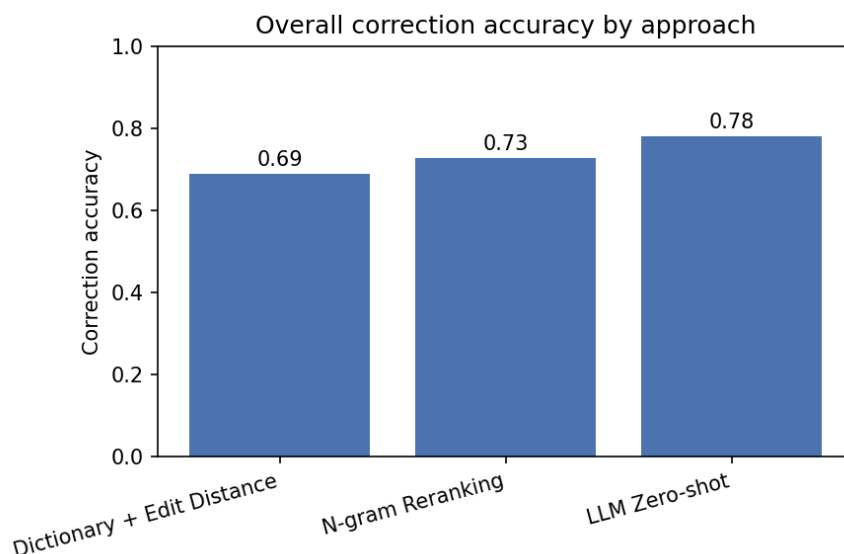
Rezultati evaluacije prikazani su najpre zbirno kroz četiri metrike na nivou celog test skupa, a zatim razloženo po tipovima grešaka. Sva tri pristupa ocenjivana su na identičnom skupu od 2.000 rečenica pa su brojevi direktno uporedivi.

5.1. Zbirni rezultati

Tabela 4 prikazuje četiri osnovne metrike za sva tri pristupa.

Pristup	Odziv detekcije	Tačnost ispravke	Stopa lažnih izmena	Rečenica/s
Rečnik + edit rastojanje	0,800	0,689	0,0199	94,98
N-gram rerangiranje	0,800	0,728	0,0199	94,65
LLM zero-shot	0,942	0,781	0,0240	2,52

Tabela 4. Zbirni rezultati tri pristupa na test skupu

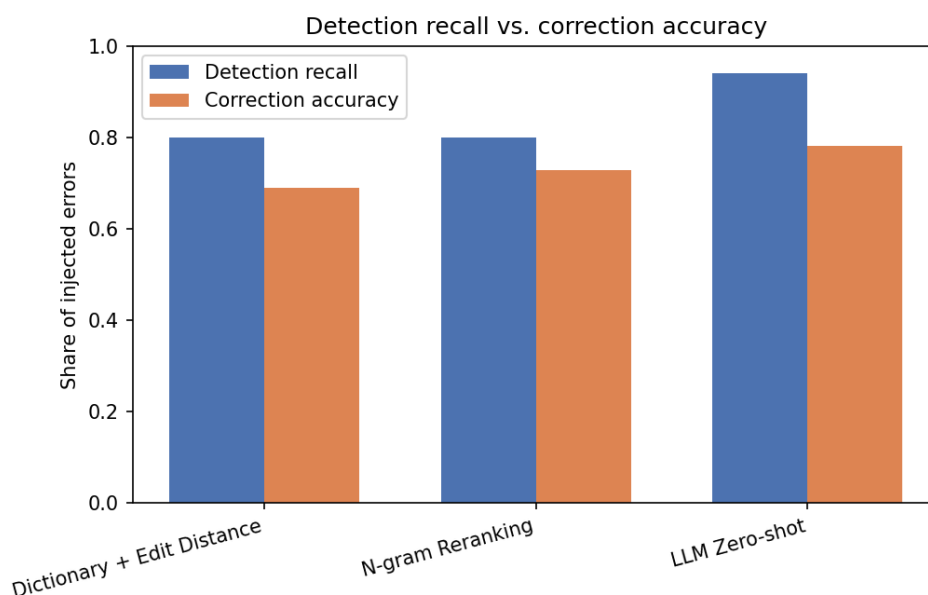


Slika 1. Ukupna tačnost ispravke po pristupu

Zbirni brojevi ocrtavaju jasnu hijerarhiju tačnosti: svaki nivo dodatne složenosti donosi merljiv dobitak. Prelazak sa unigramskog izbora kandidata na bigramsko rerangiranje podiže tačnost ispravke sa 0,689 na 0,728. Veliki jezički model dodaje još jedan skok na 0,781.

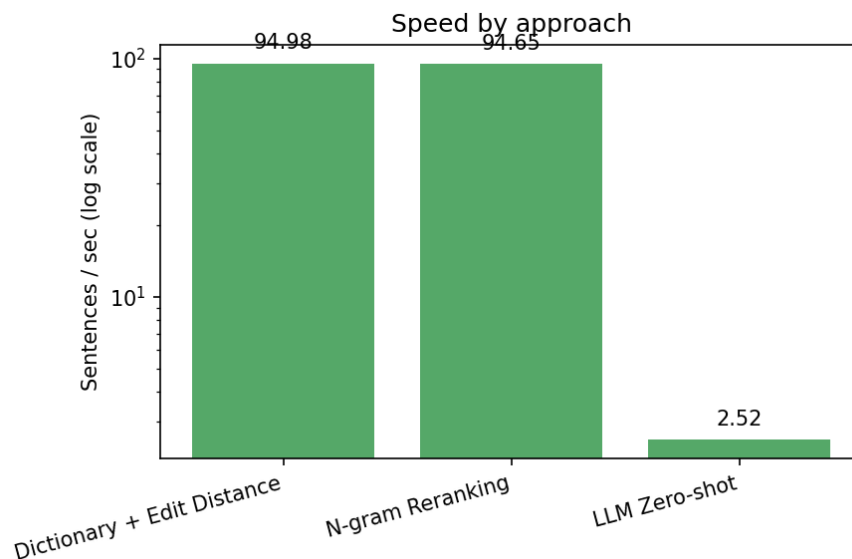
Struktura tabele otkriva i poreklo tih dobitaka. Rečnički i n-gram pristup imaju identičan odziv detekcije (0,800) i identičnu stopu lažnih izmena (0,019). To je i očekivano jer oba pristupa dele isti mehanizam detekcije. Čitava razlika u tačnosti od 3,9 procenata može se zato pripisati isključivo kvalitetnijem izboru među istim kandidatima. Odziv detekcije od 0,800 ujedno je i strukturna granica oba pristupa. Preostalih dvadeset procenata grešaka proizvodi oblike koje rečnik poznaje pa ih rečnička detekcija načelno ne može videti. To se dešava u dva slučaja: kada greškom nastane druga legitimna reč i kada se pogrešni oblici, poput očišanih varijanti, javljaju u samom korpusu iz koga je rečnik izgrađen.

Grafikon na slici 2. vizuelno pokazuje baš to: plavi stubići (detekcija) isti za prva dva pristupa, a razmak između plavog i narandžastog je "prostor" izgubljen na pogrešnom izboru kandidata.



Slika 2. Odziv detekcije naspram tačnosti ispravke

Veliki jezički model nema tu granicu: njegov odziv detekcije od 0,942 i pokazuje da kontekstualno razumevanje rečenice otkriva i veliku većinu grešaka nevidljivih rečniku. Cena je blago povećana stopa lažnih izmena (0,024 prema 0,019), jer model povremeno interveniše i tamo gde ne treba. Druga cena je brzina: oko 2,5 rečenice u sekundi prema približno 95 kod lokalnih pristupa što je razlika od gotovo 40 puta. Lokalni pristupi obrade ceo test skup za dvadesetak sekundi dok je velikom jezičkom modelu za isti posao potrebno oko trinaest minuta. Pritom izmerena brzina LLM pristupa ne odražava lokalno izračunavanje već propusnost API servisa pa zavisi od opterećenja servisa i ograničenja broja zahteva.



Slika 3. Brzina obrade po pristupu (logaritamska skala)

5.2. Rezultati po tipovima grešaka

Tabela 5 i Slika 4 prikazuju tačnost ispravke razloženu po pet tipova grešaka, po 400 grešaka svakog tipa.

Pristup	Dijakritici	Zamena	Brisanje	Umetanje	Transpozicija
Rečnik + edit rastojanje	0,667	0,698	0,393	0,912	0,775
N-gram rerangiranje	0,670	0,760	0,468	0,917	0,825
LLM zero-shot	0,830	0,700	0,750	0,830	0,795

Tabela 5. Tačnost ispravke po tipu greške

Razlaganje po tipovima pokazuje da zbirna hijerarhija krije zanimljiviju sliku: veliki jezički model, ukupno najtačniji, najbolji je na svega dva od pet tipova grešaka.

Lokalnim pristupima najlakše pada umetanje (0,912 i 0,917). Umetanje po pravilu proizvodi oblik koji nije reč pa je detekcija pouzdana, a originalna reč je gotovo uvek među kandidatima na rastojanju 1 i ubedljivo je najfrekventniji od njih. Najteže im pada brisanje (0,393 odnosno 0,468). Brisanjem znaka reč se skraćuje, a kraći oblici imaju mnogo poznatih suseda na rastojanju 1, među kojima frekvencijski kriterijum često promašuje. Upravo na brisanju bigramski kontekst donosi i najveći pojedinačni dobitak n-gram pristupa (+7,5 procentnih poena), a sličan dobitak vidi se i na zameni (+6,2) i transpoziciji (+5,0).

Profil velikog jezičkog modela je obrnut. Ubedljivo je najbolji na dijakriticima (0,830 prema 0,670) i brisanju (0,750 prema 0,468), tipovima koji zahtevaju razumevanje konteksta ili proizvode oblike nevidljive rečničkoj detekciji. Na sva tri preostala tipa, zameni, umetanju i transpoziciji, n-gram pristup ga nadmašuje (0,760 prema 0,700, 0,917 prema 0,830 i 0,825 prema 0,795). Tamo gde je greška mehanička i proizvodi nepostojeću reč pedantno nabrojanje kandidata sa kontekstnim rerangiranjem nadmašuje i model sa 70 milijardi parametara. Tamo gde je potrebno jezičko znanje i kontekst, veliki jezički model nema konkurenciju, i upravo ta dva tipa, dijakritici i brisanje, nose njegovu zbirnu prednost.



Slika 4. Tačnost ispravke po tipu greške i pristupu

5.3. Statistička značajnost razlika

Da uočene razlike u tačnosti nisu artefakt slučajnog uzorkovanja potvrđuje Meknemarov test (Tabela 6).

Poređenje	Samo prvi tačan	Samo drugi tačan	p-vrednost
Rečnik vs. N-gram	14	92	< 0,001
Rečnik vs. LLM	249	433	< 0,001
N-gram vs. LLM	266	372	< 0,001

Tabela 6. Meknemarov test: broj diskordantnih grešaka i p-vrednosti

Sva tri poređenja statistički su značajna ($p < 0,001$), pa je poredak LLM > n-gram > rečnik na nivou ukupne tačnosti pouzdan. Diskordantni parovi pritom govore i više od p-vrednosti. Kod poređenja rečničkog i n-gram pristupa asimetrija je drastična (14 prema 92). Kontekst gotovo isključivo dodaje, a retko oduzima. Kod poređenja sa velikim jezičkim modelom slika je znatno izbalansiranija, postoji veliki broj grešaka (249 odnosno 266) koje lokalni pristupi ispravljaju, a LLM ne. Pristupi nisu strogo uređeni po snazi već imaju delimično komplementarne oblasti uspeha, što otvara pitanje kombinovanih strategija.

6. Diskusija i analiza grešaka

Brojevi iz prethodnog poglavlja pokazuju koliko pristupi greše ali ne i kako. Ovo poglavlje kvalitativno analizira konkretne slučajeve uspeha i neuspeha, grupisane u četiri kategorije:

- greške koje je ispravio samo veliki jezički model
- greške na kojima se vidi doprinos konteksta n-gram modela
- greške na kojima su podbacila sva tri pristupa i slučajeve prekomernog ispravljanja.

Svi primeri potiču iz stvarnih izlaza sistema na test skupu.

6.1. Šta ume samo veliki jezički model

Greške koje je LLM ispravio, a oba lokalna pristupa promašila je da najjasnije ocrtava kvalitativnu prednost jezičkog znanja nad rečničkom mehanikom. U njoj se izdvajaju tri obrasca.

Prvi obrazac su greške koje proizvode legitimnu reč. U rečenici „Nakon rata se u seno vratilo manje od stotinu ljudi" zamena susednim tasterom pretvorila je „selo" u „seno" , postojeću reč koju rečnička detekcija po definiciji propušta. LLM iz konteksta („vratilo se u...") prepoznaje da je jedino „selo" smisljeno i ispravlja grešku koju lokalni pristupi uopšte ne vide.

Drugi obrazac su ošišani oblici koje je korpus kontaminirao. Greška „medju" (umesto „među") nije ispravljena rečničkim pristupima iz podmuklog razloga: oblik „medju" javlja se u samom korpusu dovoljno često da je uvršten u rečnik pa detekcija reč smatra poznatom. Isti mehanizam objašnjava dobar deo razlike u odzivu detekcije (0,800 prema 0,942) kod dijakritičkih grešaka. LLM, koji ne zavisi od rečnika izgrađenog iz zašumljenog korpusa, oblik bez dijakritika prepoznaje i ispravlja („zasladjuje" => „zaslađuje", „sto" => „što", „tesko" => „teško").

Treći obrazac su greške čija ispravka zahteva razumevanje šireg konteksta rečenice, poput „Stalno savetuje Ivana da ne ini greške", gde je brisanjem od „čini" nastalo „ini". Kandidata na rastojanju 1 od „ini" ima mnogo, i frekvencijski i bigramski kriterijum biraju pogrešno dok LLM iz smisla cele rečenice pogađa glagol.

6.2. Vrednost konteksta: n-gram protiv unigrama

Greške koje je n-gram model ispravio, a rečnički nije: čist je prikaz onoga što bigramski kontekst dodaje, jer se pristupi razlikuju isključivo u kriterijumu izbora među identičnim kandidatima. Obrazac je izrazito dosledan, gotovo svi primeri su morfološki promašaji unigramskog kriterijuma u kojima je frekvencijski najčešći kandidat pogrešan padežni ili glagolski oblik. Za grešku „radm" (od „radom") rečnički pristup bira frekventnije „radi", a bigram „sa radom" navodi n-gram model na tačan instrumental. Za „oblikg" (od „obliku") unigram bira nominativ „oblik", a kontekst „u obliku životinjskih" zahteva lokativ. Isti mehanizam ponavlja se kod

„crim" => „crnim" (unigram: „rim"), „praim" => „pravim" (unigram: „prvim"), „naziav" => „naziva" (unigram: „naziv") i „Braziu" => „Brazilu" (unigram: „Brazil").

Ovi primeri empirijski potvrđuju tezu da je bogata morfologija srpskog jezika prirodni teren za kontekstualne modele. Mnoštvo oblika iste lekseme na malom edit rastojanju čini unigramski kriterijum sistematski nepouzdanim, a već i minimalni kontekst od jedne susedne reči dovoljan je da se izbor dramatično popravi. Asimetrija diskordantnih parova iz Meknemarovog testa (92 prema 14) pokazuje da je ovaj dobitak gotovo besplatan jer kontekst vrlo retko pokvari izbor koji bi frekvencija pogodila.

6.3. Gde ne uspeva niko

Greške na kojima su podbacila sva tri pristupa je da mapira granice zadatka i koncentrisana je u tri klase reči.

Prvu klasu čine vlastita imena i retke reči: „Vujica" iskvareno u „Cujica", „divošku" u „duvošku", „sliper" u „sbiper". Za takve reči frekvencijski signal je preslab (javljaju se jednom ili nijednom u korpusu), kontekst ne pomaže jer se ime može javiti bilo gde, a LLM ili ne prepoznaje reč ili je ispravlja u nešto sasvim treće (za „sbiper" je predložio „Schuhber"). Ova klasa potvrđuje da su transkribovana strana imena i retke pozajmljenice sistematski najteži deo zadatka za sve pristupe.

Drugu klasu čine skraćenice: greška „tv" nastala brisanjem iz „tzv." niko nije ispravio. Oblik „tv" je i sam legitimna frekventna skraćenica, pa je greška nevidljiva i rečniku i jezičkom osećaju modela.

Treću klasu čine morfološke real-word greške: „građama" umesto „građana" legitimna je reč (dativ/instrumental množine od „grada"), pa je rečnički pristupi ne detektuju, a LLM je grešku video, ali je ispravio u pogrešan padež („građanima"). Ovi primeri pokazuju da ni kontekstualno najmoćniji pristup nije imun na finu morfološku višesmislenost srpskog jezika.

6.4. Prekomerno ispravljanje LLM-a

LLM povremeno menja i ono što nije pogrešno što se u zbirnim metrikama vidi kao povišena stopa lažnih izmena (0,024 prema 0,019). Primeri otkrivaju dva motiva. Prvi je normalizacija legitimnih varijanti: oblik „garantija" model je prepravio u standardnije „garancija", iako reč nije bila deo ubačene greške. Model ispravlja i ono što doživljava kao stilski nestandardno, prekorajući uputstvo iz prompta. Drugi je diranje reči koje model ne prepoznaje: izmišljeni pojam „metamorfmafs" model je „ispravio" u „metamorfmafs", stvorivši grešku tamo gde je nije bilo. U praktičnoj primeni ovakvo ponašanje ima realnu cenu: sistem koji menja ispravan tekst narušava poverenje korisnika čak i kada je ukupna tačnost visoka.

6.5. Praktične implikacije

Ukupna slika sugerise da izbor pristupa zavisi od scenarija primene. Za interaktivne primene sa velikim protokom teksta (provera pravopisa u uređivaču, obrada korpusa) lokalni n-gram pristup nudi najbolji odnos tačnosti, brzine i predvidivosti. On obradi 95 rečenica u sekundi nikada ne menja ono što rečnik poznaje i radi bez spoljašnjih servisa. LLM pristup sa gotovo 40 puta manjom propusnošću i zavisnošću od API servisa ima smisla tamo gde je tačnost kritična, obim teksta mali, a naročito tamo gde dominiraju dijakritičke i real-word greške, na primer pri restauraciji dijakritika u neformalnom tekstu. Komplementarnost profila (LLM najjači na dijakriticima i brisanju, lokalni pristupi na zameni, umetanju i transpoziciji) prirodno upućuje i na hibridne strategije.

7. Zaključak

U radu su implementirana i sistematski upoređena tri pristupa ispravljanja grešaka u kucanju na srpskom jeziku. Ti pristupi odgovaraju trima generacijama metoda obrade prirodnih jezika:

- rečnički pristup sa edit rastojanjem
- n-gram jezički model
- veliki jezički model u zero-shot režimu.

Poređenje je izvedeno na sintetički generisanom test skupu od 2.000 rečenica sa srpske Vikipedije sa ujednačenom raspodelom pet tipova grešaka i potpuno automatskom evaluacijom.

Rezultati potvrđuju jasnu hijerarhiju tačnosti. Svaki nivo dodatne složenosti donosi statistički značajan dobitak. Bigramski kontekst podiže tačnost ispravke rečničkog pristupa sa 0,689 na 0,728 i to gotovo isključivo boljim izborom padežnih i glagolskih oblika. Ovaj dobitak je na morfološki bogatom srpskom jeziku izraženiji nego što bi bio na engleskom. Veliki jezički model dostiže tačnost od 0,781 i jedini probija strukturnu granicu rečničke detekcije, otkriva greške koje proizvode legitimne reči, kao i očišane oblike koje je zašumljeni korpus uneo u sam rečnik.

Analiza po tipovima grešaka pokazuje da hijerarhija ipak nije apsolutna. Na mehaničkim greškama, zameni, umetanju i transpoziciji, jednostavni lokalni pristupi nadmašuju model sa 70 milijardi parametara. Veliki jezički model je najjači tamo gde je potrebno jezičko znanje, na dijakriticima i brisanju. Svoju snagu plaća prekomernim ispravljanjem ispravnih reči i brzinom manjom za gotovo 40 puta. Nijedan pristup nije univerzalno najbolji. Izbor zavisi od scenarija primene, a komplementarnost profila upućuje na hibridne strategije. Na primer brz lokalni prolaz za mehaničke greške uz pozivanje velikog jezičkog modela samo za rečenice u kojima lokalni pristup ne nalazi pouzdanu ispravku.

Rad ima i ograničenja. Test skup je sintetički: pet programski generisanih tipova grešaka ne mora verno odražavati raspodelu stvarnih ljudskih grešaka. Jezički model je bigramski, pa bi trigramski kontekst ili neuronski jezički model manjeg obima verovatno dodatno smanjio morfološke promašaje. Evaluiran je jedan veliki jezički model u jednom režimu. Dijakritički generator kandidata pokazao se kao jednostavna a delotvorna adaptacija za srpski, ali bi se detekcija očišanih oblika koje rečnik pogrešno poznaje mogla dodatno unaprediti čišćenjem korpusa ili kontekstualnom detekcijom real-word grešaka.

8. Literatura

- [1] Jurafsky, D., Martin, J. H. (2024). Speech and Language Processing, 3. izdanje. Stanford University.
- [2] Manning, C. D., Schütze, H. (1999). Foundations of Statistical Natural Language Processing. MIT Press, Cambridge, MA.
- [3] Bird, S., Klein, E., Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media, Sebastopol, CA.
- [4] Norvig, P. (2007). How to Write a Spelling Corrector. <http://norvig.com/spell-correct.html>
- [5] Leipzig Corpora Collection: Deutscher Wortschatz. <https://wortschatz.uni-leipzig.de/en>
- [6] Meta AI (2024). Llama 3.3 Model Card. https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/